Dashboard walkthrough — top to bottom

Tracing every feature from the pixels you see in the browser back through the JS → HTTP endpoint → backend handler → data source. Read in this order to match the on-screen flow.

---

1. Header strip

1.1 Title (`<h1>Wahoo Bay · Live Fish ID</h1>`)

Static text. No pipeline.

1.2 Live counters (`#stats`)

What you see — a horizontal strip:

> **frames: 1,685,852 · fps: 4.2 · with fish: 545,834 · inference: 38 ms · infer fps: 4.2**

Counter	Field on `/api/stats`	Where it comes from
`frames`	`frames_seen`	Worker's `FrameTap` thread increments on every grabbed RTSP frame. Cumulative since worker start.
`fps`	computed in `app.js` `computeFps` from a rolling 5 s window of `frames_seen` samples	Pure JS — derives from successive `/api/stats` polls.
`with fish`	`frames_with_fish`	Worker increments when at least one fish survives the negative-regions filter on a frame.
`inference`	`last_infer_ms`	Time in ms for the most recent `classifier.process_frame(frame_bgr)` call (detector + classifier combined).
`infer fps`	rolling-window derivation of `frames_inferred` (same computeFps helper, different key)	Pure JS again.

Pipeline: `app.js` tick (every 500 ms) → `GET /api/stats` → dashboard's `services/dashboard/app/main.py` `stats()` proxies to worker `http://127.0.0.1:8081/stats` → worker's FastAPI `/stats` endpoint reads from the in-memory `PipelineStats` object that `FrameTap` and the `InferenceClient` drain thread mutate.

---

2. Alerts banner (`#alerts-banner`)

What you see — bright red strip across the page, only when an SLO rule has fired. Examples: "pipeline_silence: no detection_events in the last 5 min · ack".

Step	Where
Source	Dashboard's `SLOChecker` async task (`services/dashboard/app/alerts.py`) runs every 30 s. Six rules in `SLO_RULES` — each queries Postgres or probes a worker/poller HTTP endpoint, returns pass/fail, and upserts into the `alerts` table.
Endpoint	`GET /api/alerts?active=true`
Backend handler	`services/dashboard/app/main.py` SELECTs from `alerts` where status='active'.
Frontend	`app.js` `tickAlerts()` (every 15 s) → `renderAlerts()` builds the banner DOM. Empty list → CSS class `hidden`.
Ack	`POST /api/alerts/{id}/ack` (write-token protected) updates the row.

---

3. Stream column (left half)

3.1 Live MJPEG image (`<img id="stream" src="/api/stream.mjpeg">`)

What you see — the underwater feed with bounding-box overlays.

Pipeline (live frames → browser):

1. **Camera (Axis PTZ)** publishes an H.264 RTSP stream at the URL in `VIDEO_SOURCE`.
2. **Worker `RTSPSource.frames()`** (`services/worker/app/sources.py`) opens the stream via `cv2.VideoCapture` with `rtsp_transport;tcp` and yields raw BGR frames.
3. **`FrameTap` thread** (`services/worker/app/pipeline.py`) does three things per frame: - encodes a raw JPEG and publishes to `live_raw` LiveBuffer - memcpy's the BGR bytes into a free /dev/shm slot from the pool - puts a metadata-only `FrameIn` (slot_idx + shape + ts) on `filled_q`
4. **Inference subprocess** (`services/worker/app/inference_proc.py`) reads from `filled_q`, reconstructs a zero-copy numpy view, runs YOLO detector + classifier + smoother + persistence + `cv2.imencode` of the annotated frame, puts a `FrameOut` on `out_queue`.
5. **`InferenceClient.drain_thread`** (parent process) reads `FrameOut`, publishes the annotated JPEG to `live_annotated` LiveBuffer.
6. **Worker FastAPI** exposes `/stream.mjpeg` — a `StreamingResponse` that sleeps on `LiveBuffer.wait_for_next` and yields each new annotated JPEG as a `multipart/x-mixed-replace` boundary.
7. **Dashboard** proxies via `services/dashboard/app/main.py` `/api/stream.mjpeg` → forwards bytes from `http://127.0.0.1:8081/stream.mjpeg` so the browser only knows about one origin.

3.2 Camera toggle (`#camera-toggle`)

Two buttons, "■ SeaHIVE" and "■ Pier". Switches which worker's stream the `<img>` points at.

Click	What happens
■ SeaHIVE	`_setActiveCamera("seahivecam")` → `<img>.src = /api/stream.mjpeg` (or `_raw` if bbox-toggle is off). Hits the **first worker** (port 8081).
■ Pier	Same but `/api/stream_pier.mjpeg` → dashboard proxy to **second worker** (port 8083).

The buttons themselves are filtered at load by `GET /api/cameras` — the dashboard checks whether `WORKER_URL_PIER` is configured; if not, the Pier button is hidden. localStorage remembers the last choice across page reloads.

3.3 Bbox toggle (`#bbox-toggle`)

"■ boxes: on" toggle. Rewrites the `<img>.src` between the annotated stream (`/api/stream.mjpeg`) and the raw stream (`/api/stream_raw.mjpeg`). Both are published by the same worker (the worker pre-encodes both — annotated by the inference subprocess, raw by the FrameTap thread).

3.4 Fallback banner (`#fallback-banner`)

> ■ **Camera is dark** — showing previously-recorded clips for visual continuity. *No new species, sightings, or detection data are being collected during this period.*

Source: the worker's `/stats` includes `autoswitch.is_dark`. `tick()` shows/hides the banner based on that field. Mechanism: `AutoswitchSource` (`services/worker/app/sources.py`) computes a rolling mean-luma; when it drops below `AUTOSWITCH_DARK_THRESHOLD` (default 25), the worker reads from `data/test_videos/*.mp4` instead of the live stream. During fallback, **no detection_events, frame_stats, or saves are written** — see `docs/data_pipeline.md` for the strict isolation rules.

3.5 Current detections list (`#current-list`)

Live "on-screen" panel showing what's in the *current* frame, with thumbnails of the model's top-3 picks per fish.

Step	Where
Source	Worker's `LiveFrame.detections_summary` — list of dicts populated by the inference subprocess each frame.
Endpoint	`GET /api/live.json` (proxied to worker `/live.json`)
Backend	Dashboard `services/dashboard/app/main.py` `live_json()` enriches each detection: `d["best_common"] = common_name(latin)` and adds `p["common"]` to each topk pick.
Frontend	`app.js` `tick()` calls `renderDetections()` every 500 ms. Display uses `speciesDisplayHtml(latin, common)` for the canonical "Common Name (G. species)" format with italic parenthetical.

---

4. Side column (right half) — top to bottom

4.1 Water quality strip (`.water-quality` / `#wq-grid`)

What you see — a row of "Water temp · DO · pH · Turbidity · Chlorophyll · Salinity" tiles.

Step	Where
Source	YSI EXO2 sonde reading → `sensor_readings` table. Written by the SenseStream poller (`services/sensestream_poller/app/main.py`).
Poller pipeline	Tries (1) Wahoo Bay API `api.wahoobay.net`, then (2) SenseStream direct, then (3) synthetic generator. `source` column distinguishes `live` vs `synthetic`.
Endpoint	`GET /api/water_quality/latest`
Backend	SELECT most-recent row from `sensor_readings` for `deployment_uri='wahoo_2'`.
Frontend	`tickWaterQuality()` (every 30 s) → `renderWaterQuality()`. The "as of HH:MM · source: live" text in `#wq-meta` comes from the row's `ts` + `source` columns.

4.2 Reef explorer (`#reef-explorer`)

A three-card panel: "Most-spotted fish", "When are fish most active?", "Reef conditions right now". Has a "hide" toggle that collapses the whole panel (state persisted in localStorage).

Card	Pipeline
Reef totals ("X species spotted today · Y total sightings · Z hours of footage")	`GET /api/visitor_stats` (bundled endpoint that batches species counts + hourly activity + latest water quality). Backend SELECTs from `species_sightings_mat` matview.
Most-spotted fish	Same `/api/visitor_stats`. Top-N species ordered by sightings count, rendered with `speciesDisplayHtml`.
When are fish most active?	Same endpoint — hourly buckets (one row per local hour-of-day) drawn as a horizontal bar chart in `#reef-hourly` (vanilla SVG, no chart library).
Reef conditions right now	Latest water quality from the same bundled payload (so it always agrees with the strip above it without a separate fetch).

Underlying table: `species_sightings_mat` — a materialised view of `species_sightings` (one row per persistent track, dominant species by weighted vote). Refreshed every 60 s by the dashboard's `MatviewRefresher` background task.

4.3 Trends panel (`#trends-panel`)

Three SVG charts plus a window selector (1 h / 24 h / 7 d / 30 d).

Chart	Endpoint	What it shows
Detection rate	`GET /api/trends/detection_rate?hours=N`	Bucketed `detection_events` per N-minute interval, plotted as a line.

Chart	Endpoint	What it shows
Top species over time	`GET /api/trends/top_species?hours=N`	Stacked-area chart of the top-N species' frame counts per hour. Legend below the chart.
Species × weather scatter	`GET /api/trends/species_x_weather?hours=N&param=<X>` (the dropdown sets `param`: air temp, pressure, rain, etc.)	Each point = an hour bucket. X axis = the chosen weather variable; Y axis = fish-rate per hour.

All three back to `detection_events` joined to `weather_readings` by hour. The dropdown remembers your last pick in localStorage.

4.4 Reports panel (`#reports-panel`) — the biggest surface

Seven tabs in a row across the top. Each tab is a `reports-tab-panel` div; only the active one has `.active` class. Switching dispatched by `tickReports()` in `app.js`.

Per-tab walkthrough:

4.4.1 Overview tab

Headline numbers + four small cards.

Element	Source
5 KPI tiles (Sightings, Species Found, Avg Water Temp, Avg Oxygen, Water Clarity)	Aggregations from `/api/events`, `/api/species_counts`, `/api/water_quality/history` for the selected window.
Hourly observations chart (`#ov-chart-obs`)	Same hourly rollup feeding the Reef explorer's "When are fish most active?" card, but bucketed by the chosen window.
Monitoring sites map (`#ov-map`)	Leaflet map (OpenStreetMap tiles) with one marker per camera. Marker positions come from `GET /api/export/camera_metadata.json` (the committed file at `data/camera_metadata.json`). Tiles loaded from `unpkg.com/leaflet@1.9.4` (declared in `<head>`).
Top species mini-list (`#ov-top-species`)	Reuses the same `/api/species/list` data the Fish Explorer renders.
Water quality trend (`#ov-wq-trend`)	`/api/water_quality/history?hours=<window>` — downsamples to ~144 points client-side.

4.4.2 Fish Explorer tab

The "species census" view. Where reviewers and ecologists pick a species to drill into.

Element	Source
4 KPIs (Unique species, Total sightings, Rare species, Top camera)	Derived client-side from `/api/species/list`
Top species bar chart (`#fe-top-species`)	Same data
Species richness over time (`#fe-chart-richness`)	`/api/trends/richness?hours=N` — unique species count per hour bucket

Element	Source
Species × habitat matrix (`#fe-habitat-matrix`)	`/api/species/habitat_matrix?hours=N` — per-species count per camera, rendered as a heatmap
Species detail table (`#fe-species-table`)	`/api/species/list?hours=N&min_frames=3` — backed by `species_sightings_mat`. Sortable; click any row to open the species-profile modal (see §6.1).

4.4.3 Celebrities tab (`#tab-celebrities`)

Card grid of named recurring animals (Bobby, Marty, Stumpy, etc.).

Step	Where
Source-of-truth	"Fish named by Chatters" tab of `assets/species.xlsx`.
Build script	`scripts/build_celebrities.py` parses the xlsx → `assets/celebrities.json` (25 entries, slug + nickname + species_common + binomial + id_mark + first/last seen + named_by).
Auto-refresh	`services/dashboard/app/sheet_poller.py` polls the sheet daily, re-runs the build, hot-reloads.
In-memory	Loaded into `state["celebrities"]` at dashboard startup + on every poller refresh.
Endpoint (list)	`GET /api/celebrities` — joins the static catalogue to `detection_corrections` to compute sightings count + canonical photo per celebrity (most-recent reviewer-tagged event's ROI URL).
Endpoint (detail)	`GET /api/celebrities/{id}` — same plus the full gallery of tagged sightings.
Frontend	`renderCelebritiesTab()` builds card grid; `openCelebrityModal(id)` shows the detail (see §6.3).

4.4.4 Water tab

Six KPIs (water temp, DO, pH, turbidity, chlorophyll, salinity) + four history charts.

- KPIs: `/api/water_quality/latest`
- Charts: `/api/water_quality/history?hours=N&max_points=120`

Both back to `sensor_readings`. Window selector at the top of the Reports panel applies.

4.4.5 Weather tab

Six KPIs + seven charts.

Element	Backing data
KPIs (Air temp, Wind, Rain total, Solar, Pressure, Cloud cover + condition text)	Latest row from `weather_readings`.
Air temp trend	`weather_readings.air_temp_c` over the window.
Wind speed trend	`weather_readings.wind_speed_avg_ms` (rendered in mph).
Rain trend	`rain_int_mm_hr`.

Element	Backing data
Solar trend	`solar_rad_wm2`.
Wind rose (`#ww-chart-winddir`)	Polar SVG chart binning `wind_dir_avg_deg` into 16 sectors × 6 speed bands. Calm-center disc shows fraction of "wind <0.5 m/s" hours.
Wind barbs time series (`#ww-chart-windbarbs`)	Each tick = one hour, drawn as a meteorological wind-barb glyph (direction + speed).
Cloud cover trend	`weather_readings.cloud_cover_pct`.

Underlying data: `weather_readings` table — populated by the SenseStream poller from Vaisala WXT-536 (station feed) + Open-Meteo (cloud/UV/dew-point) + nearest NWS METAR (human-readable condition text). Endpoint: `GET /api/weather/{latest,history}`.

4.4.6 Insights tab

4 KPIs + 4 scatter plots (rain × fish, pressure × fish, temp × fish, DO × fish).

Each scatter is rendered from `/api/trends/species_x_weather?param=<X>` — same endpoint the Trends panel uses, just with different `param` values.

4.4.7 Data Notes tab

Static HTML cards explaining the project's data model, model status, frame save policy, reviewer flow, and known caveats. Rendered from inline markup in `index.html` — no API call.

Notably the "Frame Save Policy" card is anchored at `#frame-save-policy` and is the target of the "■ why no thumbnail?" link in the Recent Events panel.

4.5 Downloads panel (`.downloads-panel`)

A grid of CSV/Parquet download buttons grouped into three sections: analysis-friendly, raw, documentation.

Button	Endpoint	Backing exporter (in `services/dashboard/app/exports.py`)
sightings	`/api/export/sightings.csv`	`export_sightings()` — streams from `species_sightings_mat`
hourly_summary	`/api/export/hourly_summary.csv`	`export_hourly_summary()`
tracks_timeline	`/api/export/tracks_timeline.csv`	`export_tracks_timeline()`
topk_long	`/api/export/topk_long.csv`	`export_topk_long()`
labeled_corrections	`/api/export/labeled_corrections.csv`	`export_labeled_corrections()` — joins `detection_corrections` to `detection_events` + saved frame paths
detections	`/api/export/events.csv`	`export_events()`
species_counts	`/api/export/species_counts.csv`	`export_species_counts()`

Button	Endpoint	Backing exporter (in `services/dashboard/app/exports.py`)
water_quality	`/api/export/water_quality.csv`	`export_water_quality()`
weather	`/api/export/weather.csv`	`export_weather()`
frame_stats	`/api/export/frame_stats.csv`	`export_frame_stats()`
saved_frames	`/api/export/saved_frames.csv`	`export_saved_frames()`
corrections	`/api/export/corrections.csv`	`export_corrections()`
alerts	`/api/export/alerts.csv`	`export_alerts()`
README.md	`/api/export/README.md`	Serves `services/dashboard/app/data/README.md` verbatim (data dictionary)
camera_metadata.json	`/api/export/camera_metadata.json`	Serves `data/camera_metadata.json` verbatim

All CSV exports use a psycopg async server-side cursor (`exports.py _stream()` helper) so a year of `detection_events` doesn't materialise in memory. Window + format selectors at the top of the panel append `?hours=N` and either `.csv` or `.parquet` to the URL.

4.6 Species seen (`#species-list`)

Ordered list of unique fish sighted in the last 24 h.

- Endpoint: `/api/species_counts?mode=sightings&hours=24`
- Backend: SELECT from `species_sightings_mat`, post-process to add `common` from `common_names.py`.
- Frontend: `tickReef()` (every 30 s) → `renderSpeciesList()`. Rendered with `speciesDisplayHtml` for the canonical italic-parenthetical format.

4.7 Recent events / Reviewer queue (`#events-list`)

The dashboard's most consequential surface. Where humans correct mis-identifications.

Layout: a header row with a reviewer name field + write-token field + reviewer chip (badge that turns green when the token is valid) + "■ why no thumbnail?" hover-popover. Below that, a scrollable list of event cards; below that, a "corrections: X today / Y total" counter line.

Per event card:

Element	Source
ROI thumbnail	`<img src="/api/detections/{event_id}/roi.jpg?size=160">` — backend looks up the event's saved frame on disk, crops the bbox with padding, returns a JPEG. Falls back to "no frame" placeholder when the event wasn't saved (≈95% aren't — only "interesting" frames are saved per `SAVE_INTERESTING_*` policy).

Element	Source
Species name	`speciesDisplayHtml(r.best_name, r.best_common)` — "Common Name (G. species)" with italic parenthetical. Shows "unknown" when the v2 classifier's top-1 was below `MIN_ACCEPT_ACCURACY=0.55`.
Accuracy chip	`fmtPct(r.best_accuracy)`.
Det chip	`fmtPct(r.det_conf)`.
Meta line	`{toLocaleTimeString} · {source_name} · frame {frame_id}`.
✓ **Looks right** button	Calls `endorseEvent(eventId)` → `POST /api/corrections` with `is_endorsement=true`, `corrected_name=best_name`.
✏ **Correct** button	Calls `toggleCorrection(eventId)` → opens an inline correction form (see modal in §6.2 for the same form's modal variant).

Pipeline of the events list:

1. Source: `detection_events` table populated by every worker iteration.
2. Endpoint: `GET /api/events?limit=40` (defaults). Backend filters out events that already have a row in `detection_corrections` (so reviewed events disappear from the queue) and joins to `saved_frames` to resolve `image_path`. Enriches each event with `best_common` (from `common_names.py`) and `topk[*].common`.
3. EVENT_CACHE on the client: `app.js` keeps the latest payload in a Map so the correction form can look up topk/bbox/image without re-querying. The cache is `set()` from each `/api/events` poll — never `clear()`-ed, so an event the user is viewing in the modal survives even if the next poll's top-40 doesn't include it.
4. Frontend tick: `tickEvents()` every 10 s.

Reviewer flow when "Correct" or thumbnail is clicked:

- Clicking the thumbnail opens the ROI lightbox (§6.2).
- Clicking "Correct" opens the inline correction form in place. Same fields as the modal's form (Type the correct species name + Top-K alternatives + "Tag as celebrity" dropdown + ✗ Not a fish + confidence + notes + cancel/save).
- On submit, `submitCorrection(eventId)` POSTs to `/api/corrections`. Backend validates (write-token + payload shape + celebrity_id is in the loaded catalogue) and INSERTs into `detection_corrections` with all the fields plus optional `corrected_bbox` (from the bbox editor) and optional `celebrity_id`.
- On success, the client removes the event from EVENT_CACHE and animates the list item out.

Reviewer chip (`#reviewer-chip`): rendered from localStorage values. Turns green ("reviewer: Dan") when both `reviewer-input` and `write-token-input` have content. Without the token, mutation endpoints return 401.

"■ why no thumbnail?" popover: custom hover popover. When clicked, jumps to the Data Notes tab and scrolls to the "Frame Save Policy" card (`openFrameSavePolicy()` in app.js).

4.8 Corrections stats line (`#corrections-stats`)

Single text line under the events list: "corrections: 142 today / 1,089 total".

- Endpoint: `/api/corrections/stats`
- Backend: COUNT(*) FROM detection_corrections with day-bucket grouping.
- Frontend: `tickCorrectionStats()` (every 10 s).

4.9 Input drift panel (`.drift-panel`)

Three mini-cards monitoring how the input feed's brightness + colour + detection-rate are drifting vs a 7-day and 28-day baseline.

Element	Source
Luma delta	`frame_stats_drift` view — current 1 h mean vs 7 d / 28 d mean.
Detection rate ratio	Same view — current vs baseline fish-per-frame rate.
Meta line ("as of N rows · ms")	Sample count + query latency.

- Endpoint: `/api/drift/summary`
- Backend: SELECT from `frame_stats_drift` view (defined in `db/init.sql`).
- Frontend: `tickDrift()` (every 30 s). Empty for the first ~24 h after a fresh start (drift baseline needs accumulated samples).

---

5. Modals (overlays — not in normal flow but reachable from the side column)

5.1 Species profile modal (`#species-modal`)

Opens when a row in the Fish Explorer species table is clicked.

Element	Source
Header (common + italic Latin)	Click event passes `r.latin` + `r.common`.
Summary stats line	Same source.
Hour-of-day chart	`/api/species/{species_id}/hod?hours=N` — hourly bucket counts.
Accuracy distribution chart	`/api/species/{species_id}/accuracy_dist` — histogram of `best_accuracy` across recent events.
Sample frames grid	`/api/species/{species_id}/samples` — list of `/api/detections/{event_id}/roi.jpg` thumbnails (most-recent N events with saved frames).

5.2 ROI lightbox (`#roi-modal`)

Opens when a thumbnail in the Recent Events list is clicked.

Three view tabs at the top:

Tab	Source
ROI	`GET /api/detections/{event_id}/roi.jpg?size=1024` — bbox crop with padding.
Full frame	`GET /api/detections/{event_id}/frame.jpg?size=1600&annotate=1` — whole frame with the bbox drawn.
✎ **Edit bbox**	`GET /api/detections/{event_id}/frame.jpg?annotate=0&size=2400` rendered as the background of an SVG canvas. Reviewer drags corner / edge handles or the box itself to redraw. The new bbox is stored in `_bboxEdit.current` (client-side) and travels with whatever correction the reviewer submits next.

Below the image: a copy of the correction form (same as the inline one in the Recent Events list), so the reviewer can submit without scrolling. Submit paths:

- `endorseEvent(eventId)` — POST `is_endorsement=true`
- `submitNotAFish(eventId)` — POST `not_a_fish=true`
- `submitCorrection(eventId)` — POST `corrected_name + corrected_species_id + confidence + notes`

All three include `corrected_bbox: _maybeBbox(eventId)` (if the reviewer redrew the box) and `celebrity_id: _maybeCelebrity(eventId)` (if they tagged a celebrity).

Backend: `POST /api/corrections` validates and INSERTs into `detection_corrections`.

5.3 Celebrity detail modal (`#celeb-modal`)

Opens when a card in the Celebrities tab is clicked.

Element	Source
Hero photo	`photo_url` from `/api/celebrities/{id}` — the most-recent reviewer-tagged event's ROI at `size=640`.
Nickname header	Celebrity slug → catalogue lookup.
Species line	`speciesDisplayHtml(binomial, common)`.
ID-mark description	From `assets/celebrities.json`.
Metadata DL	named_by + first_seen + last_seen + reviewer-tagged sightings count.
Gallery thumbnails	List of tagged events from `/api/celebrities/{id}` (LEFT JOIN to events so deleted events still appear, gracefully). Clicking a thumbnail jumps into the ROI lightbox for that event.

How everything ticks

The dashboard polls in different cadences (all in `app.js`):

Tick	Interval	What it refreshes
`tick()`	500 ms	`/api/live.json` + `/api/stats` → live counters, current detections
`tickHistory()`	5 s	(legacy timeseries; mostly displaced by Reports tabs)
`tickWaterQuality()`	30 s	Water quality strip
`tickDrift()`	30 s	Drift panel
`tickAlerts()`	15 s	Alerts banner
`tickCorrectionStats()`	10 s	Corrections stats line
`tickReef()`	30 s	Species seen list + Reef explorer cards
`tickTrends()`	60 s	Trends panel
`tickReports()`	on tab switch + on window selector change	Active Reports tab content
`tickEvents()`	10 s	Recent events list (reviewer queue)

All read-only endpoints are open; mutation endpoints (the correction POST, alert ack, admin endpoints) require `Authorization: Bearer <DASHBOARD_WRITE_TOKEN>` when that env var is set. The dashboard's `auth_state` chip shows whether write actions will succeed.

Quick reference: which file owns which surface

If something looks wrong with…	Look in…
Header counters / live overlay	Worker `/stats` + `/live.json` + `/stream.mjpeg` (services/worker/app/main.py)
Alerts banner	services/dashboard/app/alerts.py (SLO_RULES)
Water quality strip / Water tab	sensestream_poller + sensor_readings table
Weather KPIs / Weather tab	sensestream_poller + weather_readings table
Reef explorer / Species seen	species_sightings_mat (matview, refreshed every 60 s)
Reports → Overview map	data/camera_metadata.json + Leaflet from unpkg
Reports → Celebrities	assets/celebrities.json + sheet_poller + detection_corrections (for tag joins)
Trends + Insights scatter charts	detection_events ■ weather_readings
Downloads panel	services/dashboard/app/exports.py (one streamer per resource)
Recent events / reviewer queue	detection_events + detection_corrections + saved_frames
Drift panel	frame_stats_drift view
Species profile modal	services/dashboard/app/main.py /api/species/{id}/* endpoints

If something looks wrong with…	Look in…
ROI lightbox + bbox editor	services/dashboard/app/static/app.js `_enterBboxEdit` + `/api/detections/{event_id}/frame.jpg?annotate=0`
Celebrity detail modal + galleries	services/dashboard/app/main.py /api/celebrities/{id}
/admin/regions.html	services/dashboard/app/templates/regions_admin.html (SSH-tunnel only; not linked from the main page)

If a new feature has to land, this is the order I'd follow: data lands in Postgres (worker or poller) → SQL view or matview → dashboard endpoint in `main.py` → exporter in `exports.py` if it's downloadable → tab/panel in `templates/index.html` → renderer in `app.js` → tick in the tick table above.